

Taller — Complejidad Algorítmica y Arreglos

Asignatura: SD1004 - Estructura de Datos | **Institución:** Institución Universitaria Pascual Bravo

Docente: Juan Duque | **Semestre:** 2026-II | **Temas:** Clase 2 (Big O), Clase 3 (Arreglos 1D), Clase 4 (Arreglos 2D)

Parte 1 — Complejidad Algorítmica (Big O)

1.1 Clasifica la complejidad

1. **Buscar una palabra en un diccionario físico, abriendo siempre por la mitad de las páginas que quedan:**

$O(\log n)$.

Justificación: Al dividir las páginas a la mitad en cada paso, la cantidad de páginas restantes se reduce drásticamente de forma exponencial inversa, reduciendo el problema a la mitad en cada iteración.

2. **Revisar, uno por uno, cada carné en una caja de carnés de estudiantes hasta encontrar el que buscas:**

$O(n)$.

Justificación: En el peor caso se debe revisar individualmente cada uno de los n carnés. El tiempo total crece de manera directamente proporcional a la cantidad de carnés.

3. **Sacar la primera carta de un mazo ya barajado:** $O(1)$.

Justificación: Es un acceso directo al primer elemento. Tomar la carta superior requiere un solo paso constante, sin importar si el mazo tiene 10 o 10,000 cartas.

4. **Comparar cada estudiante de un salón con cada uno de los demás estudiantes, para ver si algún par cumple años el mismo día:** $O(n^2)$.

Justificación: Para cada uno de los n estudiantes se deben realizar n comparaciones con el resto. Esto genera un total aproximado de $n \times n$ interacciones (crecimiento cuadrático).

5. **Consultar la hora en tu reloj:** $O(1)$.

Justificación: Mirar el reloj se realiza en una única observación instantánea que no depende de ningún volumen de datos externo.

1.2 De la vida real al análisis

Tarea cotidiana: Lavar una pila de platos sucios a mano, uno por uno (tomar plato, enjabonar, enjuagar y colocar en el secador).

Efecto si la entrada (n) se hace 10 veces más grande: Si en lugar de 5 platos tengo 50 platos (10 veces más), el tiempo total requerido para lavarlos todos se multiplicará exactamente por 10, ya que cada plato requiere la misma cantidad de esfuerzo individual.

1.3 ¿Cuál escala mejor?

Recomendación: Seleccionar la **Forma B: $O(n \log n)$** .

Justificación: Aunque para conjuntos pequeños ($n = 5$) la Forma A pueda tener una constante ejecutable levemente menor, la notación Big O analiza el comportamiento a medida que n tiende al infinito. Con $n = 1,000$, la Forma A realiza **1,000,000** de operaciones (n^2), mientras que la Forma B realiza aproximadamente **10,000** operaciones ($n \log n$). Por lo tanto, la Forma B escala de manera eficiente y sostenible frente al crecimiento inesperado de datos.

1.4 Verdadero o falso (Justificado)

- a. Un algoritmo $O(1)$ siempre es más rápido en segundos reales que uno $O(n)$.

FALSO. La notación Big O mide la tasa de crecimiento, no el tiempo de reloj en segundos. Un algoritmo $O(1)$ con

una constante muy alta (ej. un proceso que tarda 10 segundos fijos) será más lento que un algoritmo $O(n)$ para $n = 1$ que toma 1 milisegundo.

b. La notación Big O describe cómo crece el tiempo de ejecución a medida que crecen los datos, no el tiempo exacto en segundos.

VERDADERO. Abstrae los detalles del hardware y se centra en la tendencia asintótica de crecimiento del algoritmo en función del tamaño de la entrada n .

c. Buscar por índice en un arreglo (`arreglo[5]`) es $O(n)$, porque hay que recorrer los primeros 5 elementos.

FALSO. Es $O(1)$. La arquitectura de computadores calcula la dirección física exacta de memoria mediante una fórmula aritmética en un solo paso, sin iterar por las casillas previas.

Parte 2 — Arreglos 1D

2.1 Diseña el arreglo

1. Representación gráfica del arreglo `notas` con sus 6 casillas:

Índice	[0]	[1]	[2]	[3]	[4]	[5]
Valor	3.5	4.2	2.8	5.0	3.9	4.5

2. **Tercera nota ingresada:** Su valor es **2.8** y se encuentra en el índice **[2]** (dado que la numeración inicia en 0).

3. **Pseudocódigo para acceso directo al último estudiante:** `nota_ultimo = notas[5]` (o en forma general, `notas[longitud - 1]`).

2.2 Direcciones de memoria

Fórmula: $\text{dirección} = \text{base} + (\text{índice} \times \text{tamaño})$. Dados $\text{base} = 0x2000$ y $\text{tamaño} = 4 \text{ bytes}$:

- `notas[0]` : $0x2000 + (0 \times 4) = 0x2000$
- `notas[3]` : $0x2000 + (3 \times 4) = 0x2000 + 12 \text{ bytes} = 0x200C$ (en hex, 12 = C)
- `notas[5]` : $0x2000 + (5 \times 4) = 0x2000 + 20 \text{ bytes} = 0x2014$ (en hex, 20 = 14)

Explicación $\$O(1)\$$: Acceder a `notas[3]` toma el mismo tiempo que a `notas[0]` porque la Unidad Aritmético Lógica (ALU) del procesador realiza la multiplicación y suma de la fórmula en un ciclo de reloj de costo constante, accediendo de forma directa a la celda en memoria RAM.

2.3 Búsqueda lineal vs. binaria

Arreglo ordenado: `edades = [15, 18, 20, 23, 27, 31, 35, 40]` . Elemento buscado: **31**.

1. Búsqueda lineal (paso a paso):

- Paso 1: `edades[0] == 15` (No)
- Paso 2: `edades[1] == 18` (No)
- Paso 3: `edades[2] == 20` (No)
- Paso 4: `edades[3] == 23` (No)
- Paso 5: `edades[4] == 27` (No)
- Paso 6: `edades[5] == 31` (¡Encontrado!)

Total: 6 comparaciones.

2. Búsqueda binaria (paso a paso):

- Paso 1: Centro entre índice 0 y 7 es $\text{floor}((0+7)/2) = 3$. `edades[3] = 23`. Como $31 > 23$, buscamos a la derecha (índices 4 a 7).
- Paso 2: Centro entre 4 y 7 es $\text{floor}((4+7)/2) = 5$. `edades[5] = 31`. ¡Encontrado!

Total: 2 comparaciones.

3. **Requisito de ordenamiento:** La búsqueda binaria requiere que el arreglo esté ordenado porque se basa en la propiedad de descartar la mitad de los datos evaluando si el valor buscado es mayor o menor que el punto medio. Si no estuviera ordenado, dicha premisa no se cumple.
4. **Para 1 millón de elementos:** Usaría la **Búsqueda Binaria**. La lineal requeriría en el peor caso 1,000,000 de comparaciones ($O(n)$), mientras que la binaria requiere a lo sumo $\log_2(1,000,000) \approx 20$ comparaciones ($O(\log n)$).

2.4 Insertar un elemento

Dada la estructura `edades = [15, 18, 20, 23, 27]`:

1. Insertar al final:

- *Si hay espacio:* Es $O(1)$, simplemente se asigna en la posición libre.
- *Si no hay espacio:* Es $O(n)$, requiere instanciar un arreglo de mayor tamaño y copiar todos los elementos existentes uno a uno.

2. **Insertar el valor 21 en la mitad:** Es un proceso costoso de orden $O(n)$. Se debe identificar la posición (índice 3) y desplazar secuencialmente los elementos mayores (`23` y `27`) una casilla a la derecha para liberar el espacio.

3. **Comparación Big O:** Insertar al final con espacio = $O(1)$; Insertar en la mitad o con redimensionamiento = $O(n)$.

Parte 3 — Arreglos 2D

3.1 Diseña la matriz

1. Representación de la matriz `salon` (3 filas × 4 columnas) con sus índices:

Fila / Columna	Columna 0	Columna 1	Columna 2	Columna 3
Fila 0	<code>salon[0][0]</code>	<code>salon[0][1]</code>	<code>salon[0][2]</code>	<code>salon[0][3]</code>
Fila 1	<code>salon[1][0]</code>	<code>salon[1][1]</code>	<code>salon[1][2]</code>	<code>salon[1][3]</code>
Fila 2	<code>salon[2][0]</code>	<code>salon[2][1]</code>	<code>salon[2][2]</code>	<code>salon[2][3]</code>

2. **Acceso a la casilla en pseudocódigo:** Asumiendo indexación basada en 0, para la fila 2 (índice 1) y columna 3 (índice 2): `puesto = salon[1][2]`. (Si se refiere textualmente al índice 2 y 3: `salon[2][3]`).

3. **Total de casillas:** $3 \times 4 = 12$ casillas. En general, se calcula multiplicando *filas* × *columnas*.

3.2 De 2D a memoria (row-major)

1. Aplanamiento continuo en memoria RAM (12 casillas secuenciales):

```
[0][0] | [0][1] | [0][2] | [0][3] | [1][0] | [1][1] | [1][2] | [1][3] | [2][0] | [2][1] | [2][2]
| [2][3]
```

2. Cálculo de la casilla [1][2] :

Fórmula: $\text{dirección} = \text{base} + ((\text{fila} \times \text{número_columnas} + \text{columna}) \times \text{tamaño})$

$\text{dirección} = 0\text{x}1000 + ((1 \times 4 + 2) \times 4) = 0\text{x}1000 + (6 \times 4) = 0\text{x}1000 + 24 \text{ bytes} = 0\text{x}1018.$

3. Cálculos adicionales:

- **[0][0]** : $0\text{x}1000 + ((0 \times 4 + 0) \times 4) = 0\text{x}1000$
- **[2][3]** : $0\text{x}1000 + ((2 \times 4 + 3) \times 4) = 0\text{x}1000 + (11 \times 4) = 0\text{x}1000 + 44 \text{ bytes} = 0\text{x}102C$

3.3 Recorrido con ciclos anidados

1. Pseudocódigo para contar puestos ocupados:

```
ocupados = 0
Para f desde 0 hasta filas - 1 Hacer:
    Para c desde 0 hasta columnas - 1 Hacer:
        Si salon[f][c] == 1 Entonces:
            ocupados = ocupados + 1
        FinSi
    FinPara
FinPara
Mostrar ocupados
```

2. **Complejidad Big O:** $O(f \times c)$ o $O(N)$ siendo N el número total de celdas. Esto se debe a que el algoritmo evalúa obligatoriamente cada casilla de la matriz una sola vez.

3. **Duplicación de dimensiones:** Si f y c se duplican ($2f$ y $2c$), el número total de casillas resulta en $2f \times 2c = 4(f \times c)$. Por consiguiente, el tiempo de ejecución **se cuadruplicará (aumenta 4 veces)**.

3.4 Conectando todo

Comprender la complejidad algorítmica (Big O) es indispensable para tomar decisiones de diseño eficientes al trabajar con estructuras de datos fundamentales. Los arreglos 1D nos brindan un acceso indexado directo **$O(1)$** gracias al cálculo directo de direcciones en la memoria RAM, pero realizar búsquedas secuenciales sobre ellos toma un tiempo lineal **$O(n)$** , el cual podemos optimizar a un tiempo logarítmico **$O(\log n)$** mediante la búsqueda binaria si mantenemos la estructura ordenada. Por otro lado, los arreglos 2D nos permiten modelar relaciones matriciales multidimensionales mapeadas de forma lineal en la memoria contigua, aunque su procesamiento global mediante ciclos anidados implica un costo proporcional al producto de sus dimensiones **$O(f \times c)$** . Dominar estos conceptos permite al desarrollador seleccionar la estructura y el algoritmo con el rendimiento computacional idóneo según el volumen y comportamiento proyectado de los datos.